

Practice Question Set :

1. What is recursion?
2. What are the two mandatory logical cases required in a recursive function?
3. What is a base case, and what is its primary purpose?
4. What is a recursive case, and how should its arguments change with each call?
5. What role does the system's call stack play during a recursive function execution?
6. What is a stack frame?
7. What is "Stack Overflow," and what causes it in recursion?
8. What are the three engineering steps to approach and solve a recursive problem?
9. In the context of recursion execution, what are the "winding" and "unwinding" phases?
10. Mathematically define the recursive relationship and base case for calculating the factorial of a number $n!$.

Important Interview/Viva Questions

11. What is the fundamental difference between an iterative solution (using loops) and a recursive solution?
12. Why is a base case absolutely essential in a recursive function? What happens if it is omitted?

13. Can a recursive function have more than one base case? Explain with an example (like the Fibonacci sequence).
14. How does the computer remember where to return after a recursive call hits its base case?
15. Explain how memory is managed during the winding and unwinding phases of $\text{fact}(4)$.
16. "Recursion is elegant but can be memory-intensive." Explain why this statement is true using the concept of stack frames.
17. What does it mean to "assume the function exists" when designing a recursive algorithm?
18. How does a recursive function move a problem closer to resolution with each step?
19. Given the summation function formula $f(n) = n + f(n-1)$, what is the direct value returned when the base case is reached at $n=1$?
20. Which data structure is implicitly used by the operating system to manage recursion?

Answers :

1. What is recursion?

Answer: Recursion is a programming technique where a function solves a problem by calling a copy of itself to work on a smaller, slightly simpler version of the original problem.

2. What are the two mandatory logical cases required in a recursive function?

Answer: To prevent infinite execution and system crashes, a recursive function must have:

Base Case (Stop): The termination condition.

Recursive Case (Reduce): The expression where the function calls itself.

3. What is a base case, and what is its primary purpose?

Answer: The base case is the termination condition of a recursive function. Its primary purpose is to stop the recursion by returning a direct value without making any further self-calls, thereby preventing a Stack Overflow.

4. What is a recursive case, and how should its arguments change with each call?

Answer: The recursive case is the part of the function where it invokes itself. With each progressive self-call, the arguments passed must reduce or break the larger problem down into a smaller, more digestible sub-problem that moves closer to the base case.

5. What role does the system's call stack play during a recursive function execution?

Answer: The call stack acts as the system memory allocator that tracks execution steps. Every time a recursive call is made, the current execution state is paused, and a new frame is pushed onto the system's call stack memory to keep track of the incomplete operations.

6. What is a stack frame?

Answer: A stack frame is an allocated block of system call stack memory created for a specific function call. It stores the function's current execution state, operations, and variables, remaining open until a resolution condition (like a return statement) is met.

7. What is "Stack Overflow," and what causes it in recursion?

Answer: Stack Overflow is a severe system crash that occurs when the call stack runs out of memory. In recursion, it is caused by a function running indefinitely without hitting a valid termination condition (base case), continuously pushing new frames onto the stack until memory limits are exceeded.

8. What are the three engineering steps to approach and solve a recursive problem?

Answer:

Assume the Function Exists: Believe that the function is successfully created and already knows how to compute smaller subsets of the problem.

Formulate the Recursive Case: Define the mathematical or logical relationship between the main problem $f(n)$ and the sub-problem $f(n-1)$.

Establish the Base Case: Determine the absolute smallest input where the answer is known instantly without any further math.

9. In the context of recursion execution, what are the "winding" and "unwinding" phases?

Answer:

Winding Phase (Going Down): The phase where the execution branches downward, pausing each step and building up stack frames until it reaches the base case.

Unwinding Phase (Returning Up): The phase where the base case resolves, and the execution flows back upward, resolving the paused frames step-by-step to calculate the final aggregated result.

10. Mathematically define the recursive relationship and base case for calculating the factorial of a number (n!).

Answer:

Recursive Relationship: $\text{fact}(n) = n * \text{fact}(n-1)$

Base Case Condition: If $n == 0$, the value is mathematically predefined as 1.

Important Interview Answers :

11. What is the fundamental difference between an iterative solution and a recursive solution?

Answer: An iterative solution uses loops (for, while) to repeat a block of code within a single stack frame, whereas a recursive solution achieves repetition by having a function repeatedly call itself, generating multiple layered stack frames in the system memory until a base case is met.

12. Why is a base case absolutely essential in a recursive function? What happens if it is omitted?

Answer: A base case is essential because it provides an explicit exit strategy for the function. If it is omitted, the function will call itself indefinitely, continually creating new stack frames until the system memory is exhausted, resulting in a **Stack Overflow** crash.

13. Can a recursive function have more than one base case?

Answer: Yes. A recursive function can have multiple base cases if there are multiple absolute smallest inputs where the answer is known instantly. For instance, in a Fibonacci sequence, both $f(0) = 0$ and $f(1) = 1$ act as instant, direct return values that terminate further deep scaling.

14. How does the computer remember where to return after a recursive call hits its base case?

Answer: The computer relies on the **Call Stack**. Each paused recursive call leaves a stack frame open in a "Waiting" status. When a frame below it resolves and returns

a value, the control naturally falls back to the immediate preceding frame waiting directly above it in stack memory.

15. Explain how memory is managed during the winding and unwinding phases of fact(4).

Answer:

Winding: Frames for fact(4), fact(3), fact(2), and fact(1) are progressively pushed onto the stack, each waiting on the layer below it.

Base Hit: fact(0) is called, hits the base case, and returns 1 directly without waiting.

Unwinding: The stack frames collapse upward sequentially: fact(1) resolves to $1 * 1 = 1$, fact(2) to $2 * 1 = 2$, fact(3) to $3 * 2 = 6$, and finally fact(4) to $4 * 6 = 24$, freeing up the allocated memory at each step.

16. "Recursion is elegant but can be memory-intensive." Explain why this statement is true using the concept of stack frames.

Answer: While recursive code is highly readable and mathematically elegant, every single self-invocation creates and pushes a brand-new **stack frame** into the system memory. These frames must remain completely open and hold local variables simultaneously until the deepest base case is resolved, consuming significantly more RAM than a standard iterative loop.

17. What does it mean to "assume the function exists" when designing a recursive algorithm?

Answer: It is a conceptual leap of faith where you design the solution under the assumption that the function is already fully operational and capable of correctly solving a smaller sub-problem. This allows you to focus purely on establishing how the main problem $f(n)$ relates structurally to that sub-problem $f(n-1)$.

18. How does a recursive function move a problem closer to resolution with each step?

Answer: It achieves this by altering the arguments passed into the progressive self-call. By subtracting from or dividing the input data (e.g., passing $n-1$ instead of n), the problem size shrinks dynamically with every call until it matches the exact value defined by the base case.

19. Given the summation function formula $f(n) = n + f(n-1)$, what is the direct value returned when the base case is reached at $n=1$?

Answer: When $n == 1$, the base case condition triggers an instant statement: return 1;. This terminates further deep scaling and stops the function from invoking itself again.

20. Which data structure is implicitly used by the operating system to manage recursion?

Answer: The operating system implicitly utilizes a **Stack** data structure (specifically the system call stack). It operates on a Last-In, First-Out (LIFO) basis, meaning the last recursive call pushed onto the stack is the very first one to be resolved and popped off once the base case is triggered.